S2D-03: SPL to DQL Translation

> **Series:** S2D | **Notebook:** 3 of 9 | **Created:** January 2026 | **Last Updated:** 01/30/2026

Overview

This notebook provides a comprehensive guide for translating Splunk SPL (Search Processing Language) queries to Dynatrace DQL (Dynatrace Query Language). While both languages share similar concepts, their syntax and capabilities differ significantly.

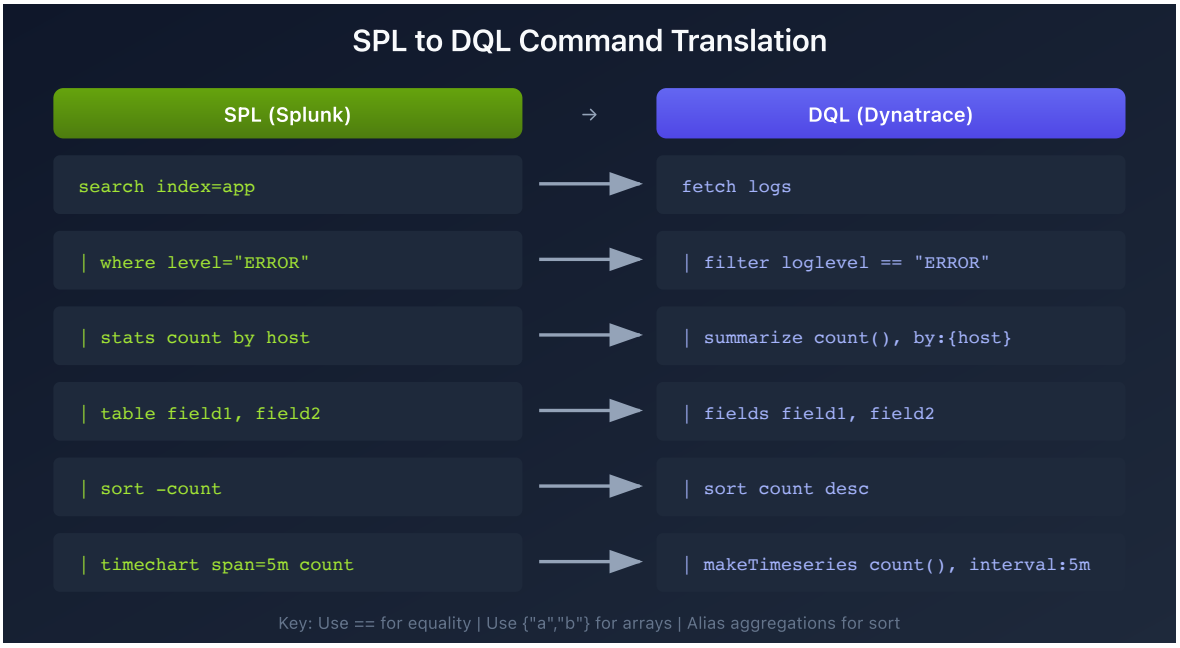

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS or Managed with Grail
Permissions	`logs.read`
Knowledge	Familiarity with source SPL queries

Learning Objectives

By the end of this notebook, you will be able to:

1. Map SPL commands to their DQL equivalents
2. Translate common SPL patterns to DQL
3. Handle differences in string matching and filtering
4. Convert aggregation and time-series queries
5. Apply DQL best practices during translation

Command Translation Reference

Core Commands

SPL Command	DQL Command	Description
`search index=...`	`fetch logs`	Select data source
`where field="value"`	`filter field == "value"`	Filter records
`stats count by field`	`summarize count = count(), by:{field}`	Aggregate data
`table field1, field2`	`fields field1, field2`	Select columns
`sort -count`	`sort count desc`	Sort results
`head 10`	`limit 10`	Limit results
`eval new=field1+field2`	`fieldsAdd new = field1 + field2`	Calculate fields
`rename old AS new`	`fieldsRename new = old`	Rename fields
`dedup field`	Not directly available	Use `summarize ... by: {field}`

String Matching

SPL Pattern	DQL Pattern	Description
`field="*error*"`	`matchesPhrase(field, "error")`	Contains text
`field=error`	`field == "error"`	Exact match
`field IN ("a","b")`	`in(field, {"a", "b"})`	Multiple values
`rex field=f "(?<name>...)"`	`parse f, "DATA:name"`	Extract patterns

Basic Query Translation

Example 1: Simple Log Search

SPL:

```
index=application sourcetype=app_logs host=app-server-01 | head 100
```

DQL:

```
// Simple log search with host filter
fetch logs, from:-1h
| filter matchesPhrase(host.name, "app-server-01")
| limit 100
```

Example 2: Error Log Count by Host

SPL:

```
index=application level=ERROR | stats count by host | sort -count
```

DQL:

```
// Error log count by host
fetch logs, from:-1h
| filter loglevel == "ERROR"
| summarize count = count(), by:{host.name}
| sort count desc
```

Example 3: Time-Based Aggregation

SPL:

```
index=application | timechart span=5m count by level
```

DQL:

```
// Time-series count by log level
fetch logs, from:-24h
| makeTimeseries count = count(), by:{loglevel}, interval:5m
```

Filtering Patterns

Phrase Matching (Contains)

SPL:

```
index=application "connection timeout"
```

DQL:

```
// Search for phrase in log content
fetch logs, from:-1h
| filter matchesPhrase(content, "connection timeout")
| limit 50
```

Multiple Value Filter (IN)

SPL:

```
index=application level IN ("ERROR", "WARN", "FATAL")
```

DQL:

```
// Filter for multiple log levels
fetch logs, from:-1h
| filter in(loglevel, {"ERROR", "WARN", "FATAL"})
| summarize count = count(), by:{loglevel}
```

NOT Filter

SPL:

```
index=application NOT level="DEBUG"
```

DQL:

```
// Exclude DEBUG level logs
fetch logs, from:-1h
| filter loglevel != "DEBUG"
| summarize count = count(), by:{loglevel}
```

Combined Conditions (AND/OR)

SPL:

```
index=application (host="app-01" OR host="app-02") AND level="ERROR"
```

DQL:

```
// Combined AND/OR conditions
fetch logs, from:-1h
| filter (matchesPhrase(host.name, "app-01") OR matchesPhrase(host.name,
"app-02"))
| filter loglevel == "ERROR"
| limit 100
```

Aggregation Patterns

Multiple Aggregations

SPL:

```
index=application | stats count, avg(response_time), max(response_time) by
host
```

DQL:

```
// Multiple aggregations by host
fetch logs, from:-1h
| filter isNotNull(response_time)
| summarize
    count = count(),
    avg_response = avg(response_time),
    max_response = max(response_time),
    by:{host.name}
```

Conditional Count

SPL:

```
index=application | stats count(eval(level="ERROR")) as errors, count as
total by host
```

DQL:

```
// Conditional count - errors vs total
fetch logs, from:-1h
| summarize
    errors = countIf(loglevel == "ERROR"),
    total = count(),
    by:{host.name}
| fieldsAdd error_rate = (toDouble(errors) / toDouble(total)) * 100
```

Field Extraction

Regex Pattern Extraction

SPL:

```
index=application | rex field=_raw "user=(?&lt;username&gt;\w+)"
```

DQL:

```
// Extract username from log content
fetch logs, from:-1h
| filter matchesPhrase(content, "user=")
| parse content, "LD 'user=' WORD:username"
| summarize count = count(), by:{username}
| sort count desc
```

Key-Value Extraction

SPL:

```
index=application | rex field=_raw "status=(?&lt;status&gt;\d+)" | rex
field=_raw "duration=(?&lt;duration&gt;\d+)"
```

DQL:

```
// Extract multiple fields from structured log
fetch logs, from:-1h
| parse content, "LD 'status=' INT:status LD 'duration=' INT:duration"
| filter isNotNull(status) AND isNotNull(duration)
| summarize avg_duration = avg(duration), by:{status}
```

Time-Series Queries

Timechart Equivalent

SPL:

```
index=application level=ERROR | timechart span=1m count by host
```

DQL:

```
// Time-series error count by host
fetch logs, from:-24h
| filter loglevel == "ERROR"
| makeTimeseries count = count(), by:{host.name}, interval:1m
```

Key Syntax Differences

Important DQL Rules

Aspect	SPL	DQL
String quotes	Single or double	Double only
Array syntax	`("a", "b")`	`{"a", "b"}`
Equality	`=`	`==`
Named params	Positional	Required: `decimals: 2`
Pipe symbol	`|`	`|`
Field access	`field_name`	`field.name` (nested)

Aggregation Aliasing (Critical)

In DQL, aggregations MUST be aliased if you want to reference them later:

```
// Wrong - cannot sort by count()
| summarize count(), by:{host.name}
| sort count() desc  // ERROR!

// Correct - alias the aggregation
| summarize count = count(), by:{host.name}
| sort count desc  // Works!
```

Next Steps

With your queries translated, proceed to **S2D-04: Alert Migration - Davis Anomaly Detectors** to learn how to convert Splunk alerts to continuous Dynatrace monitoring.

References

- DQL Reference
- DQL Functions

- Parse Command

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.